

Evaluating Planning Abilities Of Single-Agent And Multi-Agent LLM Frameworks

Sri Harsha Gollamudi¹, Samavedam Manikhanta Praphul¹, Poorna Chandra Vemula¹

¹Northeastern University

Abstract

Large language models (LLMs) have demonstrated impressive capabilities in natural language tasks but face significant challenges in complex planning scenarios. This study evaluates the planning abilities of state-of-the-art LLMs in structured game environments (Blocksworld, Barman, and Grippers) using rigorous metrics focused on state interpretation, constraint adherence, goal achievement, and solution optimality. We compare ten advanced single-agent LLMs including GPT-4, Claude 3 Opus, and Gemini models against three novel multi-agent frameworks: debate-based, sequential, and hierarchical. Our experiments across 20 carefully crafted prompts per game reveal that single-agent approaches struggle with consistent plan generation, with most models showing high failure rates in constraint adherence and optimal planning. The multi-agent frameworks demonstrate marked improvements, particularly in start state interpretation and constraint adherence, with the debate framework achieving up to 95% success in goal state achievement compared to baseline performance. This work provides valuable insights into the current limitations of LLM planning capabilities and demonstrates the potential of multi-agent approaches for enhancing planning performance in structured environments.

1 Introduction

Large language models (LLMs) have made remarkable progress in recent years, demonstrating impressive capabilities across a wide range of natural language tasks. However, despite their advancements in areas like text generation, question answering, and language understanding, LLMs still face significant limitations when it comes to complex planning and reasoning tasks. Our research reveals that current LLMs struggle to generate coherent, step-by-step plans for achieving specific goals, especially in structured environments with

defined rules and constraints. This limitation becomes particularly evident when LLMs are tasked with planning in game-like scenarios that require precise, sequential decision-making.

To address these shortcomings, our paper explores the potential of agentic frameworks that leverage multiple models, moving beyond the limitations of single-model approaches. We investigate how collaborative multi-agent systems, such as multi-agent debates, can enhance planning capabilities. Our research aims to demonstrate that these agentic frameworks often outperform traditional single-model baselines by combining diverse perspectives and leveraging the strengths of multiple specialized models. This approach allows for more robust and effective planning in complex environments.

To evaluate the planning abilities of both single-model and multi-agent systems, we used a comprehensive dataset based on classic planning domains and games. This dataset includes scenarios from environments such as Blocksworld, Barman, Floor-tile, Grippers, Advanced Grippers, Tyreworld, Advanced Tyreworld, Termes and Storage. However, we will be considering only a subset comprising of three games i.e. Blocksworld, Barman and Grippers in our study due to time and budget constraints. We also established a set of rigorous metrics to assess model performance. These metrics evaluate crucial aspects of planning, including the ability to reach the goal state, plan optimality, accurate interpretation of start and goal states, and adherence to environmental constraints. By applying these metrics to various model configurations, our study provides valuable insights into the current state of LLM planning capabilities and the potential for improvement through multi-agent approaches.

Our study utilizes a comprehensive dataset that builds upon existing planning domains, specifically designed to challenge and evaluate the planning

capabilities of LLMs. The dataset comprises 20 carefully crafted prompts for each game environment, including Blockworld, Barman, and Grippers. These prompts are designed to explore various scenarios within each game, providing a diverse range of planning challenges.

Each prompt in our dataset is structured to provide clear and unambiguous information about the initial state, goal state, available actions, and specific constraints of the environment. This standardized format ensures a consistent and fair evaluation of the models’ planning abilities across different scenarios. The prompts are specifically designed to test not only the models’ understanding of the game rules but also their capacity to reason about optimal sequences of actions to achieve the desired outcomes. The dataset, which forms the foundation of our study, is publicly available at <https://github.com/VITA-Group/o1-planning/tree/main>.

2 Related Work

LLMs have demonstrated remarkable capabilities across diverse tasks, including language-related tasks such as problem solving, code generation, ethical reasoning, medical usage, agentic applications and education (Chang et al., 2024).

Of particular interest is their potential for planning in interactive physical environments, such as robotics and gaming (Huang et al., 2022; Song et al., 2023; Lin et al., 2023).

Despite advances in prompting techniques like Chain-of-Thought (CoT) (Wei et al., 2022) and Tree-of-Thoughts (ToT) (Yao et al., 2024), LLMs have historically struggled with generating feasible plans.

However, recent architectural breakthroughs in models such as GPT-4 O1, Claude 3.5, and Llama 3 405B have demonstrated unprecedented performance in code generation and problem-solving tasks, suggesting potential improvements in planning capabilities as well. (Anthropic, 2024a; OpenAI, 2024b; Dubey et al., 2024)

Recent research has evaluated single-agent planning abilities using a variety of games represented as text prompts. The findings indicate that single LLM agents struggle with planning in environments that involve numerous actions and constraints. Building on this baseline, we propose leveraging multi-agent frameworks to enhance planning capabilities in the same game environ-

ments explored in the study (Wang et al., 2024).

The emergence of multi-agent frameworks has shown promise in enhancing LLM performance, particularly in areas requiring factual accuracy and complex reasoning (Du et al., 2023; Liu et al., 2024).

These approaches, which leverage debate and collaboration between multiple AI agents, have consistently achieved state-of-the-art (SOTA) results across various benchmarks. Further, there are some works which demonstrated the use of multi-agent frameworks particularly for planning in robotics domain. (Mandi et al., 2024). This success motivates our investigation into applying multi-agent systems to planning scenarios in the games environments.

3 Methodology

Our current approach involves human evaluation of the selected models against our defined metrics. At present, this approach involves manual prompting each of the selected LLMs for each of the designed prompts for all of the games selected. Our approach provides the human ground evaluation of the these model’s performance in the selected games against these designed prompts involving several scenarios. We shall be considering a simple average of the model performance in our metrics across different prompts of the games to measure the model performance.

3.1 Metrics

To evaluate the performance of LLMs in planning tasks across various game environments, we developed a set of comprehensive metrics. These metrics are designed to assess the models’ ability to interpret the problem, adhere to constraints, and generate effective plans. Here, we define and expand on each of these metrics.

3.1.1 Start State Misinterpretation

This metric evaluates whether the model correctly understands and uses the given initial state of the game environment. A misinterpretation occurs when the model bases its plan on an incorrect assumption about the starting conditions. Accurate interpretation of the start state is crucial for developing a valid plan. Misinterpretation can lead to plans that are impossible to execute or fail to achieve the goal.

Examples of misinterpretation:

- In Blocksworld, assuming a block is on top of another when it's actually on the table.
- In Barman, assuming a shaker is clean and empty when it actually contains a mixed cocktail in the initial state.

3.1.2 Goal State Misinterpretation

This metric assesses the model's comprehension of the desired end state. A misinterpretation here means the model is working towards an incorrect goal. Correctly understanding the goal state is essential for generating a plan that actually solves the given problem. Misinterpretation here can result in plans that, even if executed perfectly, fail to achieve the desired outcome.

Examples of misinterpretation:

- In Floortile, aiming to paint tiles in the wrong color pattern.
- In Grippers, planning to move objects to incorrect rooms.

3.1.3 Constraint Violation

This metric identifies instances where the model's plan includes actions that are not allowed within the game's rules or lead to invalid states. Adhering to constraints is crucial for generating feasible plans. Violations indicate a fundamental misunderstanding of the game's mechanics or a failure in logical reasoning.

We categorize these violations into two types:

Illegal moves: Actions that are not permitted by the game's rules.

Illegal states: Sequences of actions that result in a game state that should not be possible.

Examples of Constraint Violation:

- In Tyreworld, attempting to remove a wheel without first loosening the nuts (illegal move)
- In Blocksworld, planning a state where a block is simultaneously on top of two other blocks (illegal state)

3.1.4 Reached Goal State

This metric evaluates whether the model's plan, if executed, would successfully achieve the desired goal state without any of the aforementioned errors.

This is the primary measure of the model's overall planning ability. It indicates whether the model can integrate its understanding of the start state, goal state, and constraints to produce a valid solution.

3.1.5 Optimality

For plans that successfully reach the goal state, this metric assesses whether the solution is optimal, defined as using the minimum number of actions necessary assuming no parallel execution. While reaching the goal state is the primary objective, the efficiency of the solution is also a critical factor in evaluating planning capabilities. Optimal plans demonstrate not just the ability to solve the problem, but to do so in the most efficient manner possible.

By applying these metrics to the outputs of various LLMs across different game environments, we can comprehensively evaluate their planning capabilities. This approach allows us to identify specific strengths and weaknesses in the models' performance, providing insights into areas for improvement in LLM-based planning systems.

3.2 Single-Agent Framework (Baseline)

The Single-Agent Framework serves as the baseline for evaluating the effectiveness of multi-agent approaches in planning tasks. In this framework, a single LLM is presented with a problem statement and associated constraints, and is tasked with generating a complete plan in a single pass. The model processes the input prompt, which includes the initial state, goal state, and any relevant rules or restrictions of the environment. It then produces an output that attempts to solve the given problem. This straightforward approach relies solely on the model's inherent capabilities to interpret the problem, reason about possible solutions, and generate a coherent plan. While this method benefits from simplicity and direct application of the model's knowledge, it lacks the iterative refinement and diverse perspectives offered by multi-agent frameworks. The performance of this single-agent baseline across various metrics—such as start and goal state interpretation, constraint adherence, goal achievement, and plan optimality—provides a crucial reference point for assessing the potential improvements introduced by more complex, collaborative AI planning strategies. The baseline models are GPT-4O, GPT-4, O1-Mini, O1-Preview, Claude 3.5 Sonnet, Claude Opus, Gemini Flash 1.5, Gemini Pro 1.0, Sonar Huge, and Sonar Large.

3.3 Multi-Agent Frameworks

Despite the impressive capabilities of advanced LLMs like GPT-4 O1, Claude 3.5, and Llama 3 405B in various problem-solving tasks, they still face significant challenges when it comes to complex planning scenarios. Our research reveals that even these state-of-the-art single-model approaches struggle to consistently generate coherent and optimal plans in structured environments with multiple constraints and sequential decision-making requirements.

To address these limitations, we propose leveraging multi-agent frameworks that have shown promise in enhancing LLM performance, particularly in areas requiring factual accuracy and complex reasoning. These approaches, which utilize debate and interaction between multiple AI agents, have consistently achieved state-of-the-art results across various benchmarks, motivating our investigation into their application for planning tasks. The models used for frameworks are GPT-4O, Claude 3.5 Sonnet, Gemini Flash 1.5 and Gemini Pro 1.0.

In this study, we explore three multi-agent frameworks and evaluate their performance in planning scenarios: Multi-Agent Debate, Multi-Agent Sequential, and Multi-Agent Hierarchical Framework.

3.3.1 Multi-Agent Debate Framework

The Multi-Agent Debate Framework is an innovative approach designed to enhance the planning capabilities of large language models (LLMs) through structured interaction between multiple AI agents. This framework leverages the power of debate and critical thinking to generate more robust and effective plans in complex environments.

In this framework, two AI agents engage in a two-round debate process. During the first round, each agent independently generates an initial plan based on the given problem statement and constraints. This allows for diverse perspectives and approaches to be considered from the outset.

The second round introduces a critical evaluation phase, where each agent reviews and critiques the plan proposed by its counterpart. This step is crucial as it encourages the identification of potential flaws, oversights, or inefficiencies in the initial plans. Following this critique, both agents have the opportunity to refine and improve their original proposals, taking into account the feedback received.

The Multi-Agent Debate Framework offers several advantages over single-agent approaches in planning scenarios. Firstly, it introduces a level of diversity in problem-solving strategies that a single model might not achieve on its own. By having two agents approach the problem independently, there's a higher likelihood of exploring a broader solution space.

Secondly, the critical evaluation phase mimics human-like reasoning processes, where ideas are scrutinized and refined through discussion. This can lead to the identification and correction of errors that might go unnoticed in a single-pass planning attempt. The framework essentially creates a self-correcting mechanism within the planning process.

Lastly, the iterative nature of the debate allows for the progressive refinement of plans. As agents respond to critiques and incorporate new insights, the quality of the final plan tends to improve. This is particularly valuable in complex planning scenarios where initial solutions may overlook subtle constraints or fail to optimize for all relevant factors.

By fostering a collaborative yet critical environment, the Multi-Agent Debate Framework has the potential to produce more comprehensive, error-resistant, and optimized plans compared to traditional single-agent approaches. This makes it a promising tool for addressing the limitations observed in state-of-the-art LLMs when tackling complex planning tasks.

3.3.2 Multi-Agent Sequential Framework

The Multi-Agent Sequential Framework represents an innovative approach to enhance the planning capabilities of large language models (LLMs) through a structured, iterative process involving multiple AI agents. This framework harnesses the strengths of different agents working in sequence to refine and improve the planning process. The framework operates over one round of iteration, with three AI agents collaborating in a specific order.

Agent 1 generates an initial plan based on the given problem statement and constraints. This plan is then passed to Agent 2, who reviews, refines, and potentially improves upon it. Subsequently, Agent 2's output is sent to Agent 3 for further analysis and enhancement.

The sequential approach offers several advantages over single-agent systems in planning scenarios.

ios. Firstly, it provides diverse perspectives, as each agent may have different strengths or specialized knowledge, allowing for a more comprehensive approach to problem-solving. Secondly, the multiple passes enable iterative refinement, allowing for continuous improvement of the plan, with each agent potentially catching oversights or suggesting optimizations that others might have missed.

Error correction is another significant benefit of this framework. If one agent makes a mistake or overlooks a crucial detail, subsequent agents have the opportunity to identify and correct these issues. Additionally, the framework allows for specialization, as agents can be designed or prompted to focus on specific aspects of the planning process, such as initial ideation, constraint checking, or optimization.

The cumulative knowledge aspect of this framework is particularly noteworthy. As the plan passes through multiple agents, it accumulates insights and improvements, potentially leading to more robust and effective final plans. By leveraging the sequential interaction of multiple agents, this framework aims to produce more comprehensive, error-resistant, and optimized plans compared to traditional single-agent approaches. This makes it a promising tool for addressing the limitations observed in state-of-the-art LLMs when tackling complex planning tasks, particularly in environments with multiple constraints and sequential decision-making requirements.

3.4 Mult-Agent Hierarchical Framework

The Hierarchical Planning Framework enhances the problem-solving capabilities of large language models (LLMs) by employing a master-slave agent architecture. This structured approach divides complex tasks into manageable subtasks, with slave agents generating initial solutions independently and the master agent synthesizing these outputs into a cohesive final plan. By assigning specialized roles, the framework ensures a systematic and collaborative process for tackling intricate problems.

In our setup, we begin with slave agents addressing the problem based on a shared prompt and context. Each agent works independently, producing diverse solutions that are logged for further evaluation. This decentralized approach broadens the solution space by fostering varied perspectives and strategies.

Once the slave agents complete their tasks, the master agent critically evaluates their outputs to

ensure alignment with the problem’s constraints and objectives. Using a detailed prompt that incorporates all slave responses, the master agent synthesizes an optimized final plan by integrating the best elements of each proposal while resolving inconsistencies or inefficiencies.

This framework offers key advantages over single-agent approaches, including modularity, parallel processing, and diversity in problem-solving strategies. The master agent’s synthesis process acts as a quality control mechanism, ensuring comprehensive and refined solutions. By combining decentralized problem-solving with centralized oversight, the framework produces innovative and optimized plans, making it a robust tool for addressing complex planning challenges.

4 Experimental Setup

We have considered the games of Blockworld, Barman, and Grippers for evaluating the performance of planning of the selected LLM. As discussed early, for each of these models and for each of these games, we have explored 20 curated design prompts for these games. For the LLMs the specific instructions explaining the details of the game are provided at the start of the prompt which are provided in the appendix [A](#).

4.1 Games

In this subsection, we will have a brief about the games that we have explored in our work to provide a quick overview of the game.

4.1.1 BlocksWorld

This game involves navigating from an initial state of blocks to a goal state of blocks through four actions which represent the actions of picking up a block, putting down the lifted block, stacking the set of blocks and unstacking the set of blocks with restrictions such as you can lift only a single block at a time using an empty arm.

4.1.2 Barman

This game represents a robotic bartender system with two hands that can prepare and serve cocktails through various actions. The robot can manipulate drink dispensers, shot glasses, and a shaker to create cocktails under specific constraints. The main actions include grasping and placing containers, handling shot glasses (filling, refilling, emptying, and cleaning), managing the shaker (pouring ingredients, cleaning, shaking), and pouring finished

cocktails. Each action has precise preconditions, such as requiring empty hands or specific container states, and results in defined state changes like container contents, cleanliness levels, and hand occupancy. The goal of the game is to efficiently sequence these actions to prepare requested drinks while adhering to all operational constraints.

4.1.3 Grippers

This game involves moving objects by robots across rooms to reach final state of objects in rooms using two grippers of each robot from the initial state of objects through three actions which represents the action of picking up objects, moving across rooms and dropping the picked objects with restrictions that any object can be picked only if any gripper of the robot is free.

4.2 Models

In this section, we shall have a quick overview on the models that we are exploring in our work, more detail about the model can be understood by following the reference links provided. For our multi-agent setup, we will be utilizing the following models: Claude 3.5 Sonnet, GPT-4o, Gemini 1.0 Pro and Gemini 1.5 Flash.

4.2.1 GPT-4 Omni (GPT-4o)

GPT-4o which stands for Generative Pre-trained Transformers was OpenAI's flagship model that can process multimodal inputs and outputs such as audio, vision, and text capabilities having performance similar to GPT-4 Turbo's performance on English text and code while being faster and 50% cheaper. The model was mentioned as to provide more natural human-computer interaction across various applications. More details of the model ([OpenAI, 2024a](#))

4.2.2 Sonar Large

Sonar Large is Perplexity's large (70B) language model, part of the Llama 3.1 Sonar family having 131,072 token context window built on top of the Llama 3.1's 70B model. This model is designed to be more helpful in generating responses for text based applications such as chat, language generation, and text-to-text tasks. More details of the model ([Perplexity AI, 2024b](#))

4.2.3 Sonar Huge (405B model)

Sonar Large is Perplexity's Huge (405B) language model, part of the Llama 3.1 Sonar family having 127,072 token context window built on top of the

Llama 3.1's 405B model. Unfortunately details about this model are not revealed but only mentioned as provided in the model cards ([Perplexity AI, 2024a](#)).

4.2.4 Claude 3 Opus

Claude 3 Opus is Anthropic's largest and highly capable model with multi-modal capability. No information about model size has been provided, and though the model has support of a context window of 1M tokens, in production a context window of 200K tokens is used. Claude 3 Opus is particularly better in complex reasoning tasks and specialized capabilities compared to 3.5 Sonnet. More details of the model. ([Anthropic, 2024a](#))

4.2.5 Claude 3.5 Sonnet

Claude 3.5 Sonnet is Anthropic's model which is Anthropic's balanced model offering combination of skills and speed compared to Claude 3 Opus. Similar to Claude 3 Opus, no information about model size has been provided, and though the model has support of a context window of 1M tokens, in production a context window of 200K tokens is used. More details of the model ([Anthropic, 2024b](#)) ([Anthropic, 2024a](#)).

4.2.6 Gemini 1.5 Flash

Gemini 1.5 Flash is Google's latest AI model, designed for rapid inference and deployment. It offers significantly faster processing speeds compared to its predecessors while maintaining high-quality outputs. The model is optimized for quick responses in various applications, making it suitable for real-time interactions and large-scale deployments.

4.2.7 Gemini 1.0 Pro

Gemini 1.0 Pro is Google's advanced multimodal AI model, capable of understanding and generating text, images, audio, and video. It offers improved performance across a wide range of tasks, including natural language processing, image recognition, and complex reasoning. This model is designed to handle diverse inputs and produce coherent, context-aware outputs.

4.2.8 O1-Mini

O1-Mini is part of OpenAI's o1 series, focused on reasoning and problem-solving. It's designed for technical fields such as coding and mathematics, offering fast and efficient responses while maintaining high accuracy. O1-Mini operates with just 1.3 billion parameters and requires only 4GB of

RAM, making it accessible for organizations with limited computing power. It's particularly valuable for tasks requiring quick responses and resource-efficient processing.

4.2.9 O1-Preview

O1-Preview, also known as Strawberry, is OpenAI's cutting-edge AI model designed to excel in reasoning-intensive tasks. It utilizes Chain of Thoughts and Self Reflection methods, spending more time "thinking" before generating a response. O1-Preview has demonstrated remarkable improvements in mathematics, coding, and science tasks, often performing at levels comparable to PhD students or professional programmers.

4.2.10 GPT-4

GPT-4 is OpenAI's large multimodal model capable of processing both text and image inputs. It demonstrates human-level performance on various professional and academic benchmarks. GPT-4 is known for its broad capabilities in natural language understanding, generation, and complex reasoning tasks. While specific details about its architecture are not publicly disclosed, it represents a significant advancement in AI language models.

4.3 Prompts

The prompts that are used in the model evaluation are arranged in the increasing order of complexity like the first prompt is just about the initial model and final state with no requirements of any manipulations. Next prompt i.e. second prompt involves moving a single object i.e. initial and final game states differ by a single move or so (one manipulation). Next prompt, the complexity increases to have more than single manipulation of the environment to move from initial state to final state. More details of the prompts are present in [A](#)

5 Results and Discussions

In this section, we explore our findings of the model on the metrics we have considered across different setups.

5.1 BlocksWorld

Figure 1 shows that most models, except GPT-4o, Gemini-Pro-1.0 successfully interpret the start state in BlocksWorld. GPT-4o, Gemini-Pro-1.0 exhibited a notable failure rate, indicating potential issues with understanding or processing the initial conditions accurately.

Most multi-agent frameworks maintained perfect accuracy, with Claude 3.5 Sonnet, Debate, Hierarchical, and Sequential frameworks all achieving 100% success rates as shown in Figure 6. However, the baseline framework showed slightly lower performance, indicating that multi-agent approaches may offer advantages in interpreting block configurations.

In Figure 2 as well, all models except GPT-4o achieve perfect success rates. GPT-4o again shows some failures, highlighting a consistent issue with accurately understanding or planning towards the correct start and goal states.

The goal state interpretation results for BlocksWorld reveal notable differences between baseline and multi-agent frameworks as shown in Figure 8. All of the frameworks consistently achieved 100% accuracy, showing particular improvement over the baseline for GPT-4o implementations in particular.

Figure 4 reveals significant challenges across all models. While Claude 3 Opus and Claude 3.5 Sonnet perform relatively well, achieving moderate success rates, other models like Sonar Huge 405B and Sonar Large 70B struggle more noticeably. This indicates that translating start and goal state understanding into effective action sequences remains a complex task for many models.

The multi-agent frameworks showed marked improvements over baseline performance as shown in Figure 10. The Debate framework enhanced GPT-4o's success rate to approximately 60% while maintaining Claude 3.5 Sonnet's high performance at nearly 95%. The Sequential framework demonstrated similar improvements, achieving around 50% success rate with GPT-4o and 90% with Claude 3.5 Sonnet.

In terms of following constraints, Claude 3 Opus and Claude 3.5 Sonnet again show better performance compared to others as shown in Figure 3. However, all models display varying degrees of failure in adhering to constraints, with Sonar Large 70B showing the highest failure rate. This suggests that constraint adherence is a critical area where improvements are needed, possibly through more explicit constraint-checking mechanisms.

The constraint violation analysis reveals significant variations between baseline and multi-agent frameworks. The multi-agent frameworks demonstrate substantial improvements in constraint adherence as shown in Figure 12. The Debate framework reduces GPT-4o's constraint violations to approxi-

mately 40% while maintaining Claude 3.5 Sonnet’s low violation rate at around 10%. The Sequential framework shows similar improvements, though slightly less pronounced than the Debate approach.

Figure 5 shows that achieving optimal solutions is particularly challenging for all models, with high failure rates across the board. This reflects a general difficulty in not only reaching the goal state but doing so efficiently. The results suggest a need for improved strategies or frameworks that enhance models’ ability to generate optimal plans.

The multi-agent frameworks demonstrated varied impacts on optimality as shown in Figure 14. The Debate framework maintained GPT-4o’s high performance at around 80% optimality, while slightly improving Claude 3.5 Sonnet’s performance to approximately 95%. The Sequential framework showed the most consistent performance, achieving 100% optimality with GPT-4o and maintaining high performance with Claude 3.5 Sonnet.

5.2 Grippers

All the models except GPT-4 have failed at-least once in interpreting the state either the start or final state as evidenced in figures 1, 2. Also, we observed that the Gemini-1.0-pro has the lowest accuracy in both start state interpretation and end state interpretation.

This inability to interpret the state properly has also caused the models to have illegal state during the planning causing constraint violations, but Claude 3 Opus, O1-preview appears not fall in this trap as clearly depicted in 3 thereby increasing its ability to reach the goal state.

Based on figure 4, except Gemini-1.0-pro, all LLMs have reached goal state with 40% success rate. Lastly, based on figure 5, the model O1-mini has always provided the optimal solution on feasible plan generation. Apart from GPT-4, Gemini-1.0-pro, all other LLMs have provided at-least 50 % of times optimal plan.

As evidenced in figure 6, frameworks enhance the systems to have 100% start state interpretation accuracy, while the goal state can improve but not guaranteed as shown in figure 10. On the metric of constraint violation, the framework ‘Sequential’ has remained the same or worsen as captured in figure 12, however other frameworks have improved system by decreasing the constraint violation. For metrics of reaching goal state and generating optimal plans, the frameworks have overall contributed

in improving the performance as observed in figures 10, 14. More importantly the optimal accuracy was mostly improved in the debate framework as observed in table 15.

5.3 Barman

In the Barman game, there are approximately ten actions, each with specific restrictions based on the environment’s state. This complexity presents a significant challenge for LLMs attempting to plan in this game environment.

In the Barman game environment, as illustrated in figures 1 6 all evaluated models demonstrated perfect accuracy in start state interpretation, achieving a 100 percent success rate across all the cases in both single-agent and multi-agent framework setups. This consistent performance indicates robust capability in understanding initial game configurations despite the game’s complexity involving multiple actions and state-dependent restrictions.

Based on figures 2 8, Goal state interpretation performance reveals interesting patterns across different models. Interestingly, O1-preview and O1-mini fail in understanding the goal state in 30 percent cases. Other models demonstrate exceptional accuracy, achieving nearly 100 percent success in correctly interpreting goal states both in single-agent and multi-agent setup. This high performance indicates their robust understanding of complex drink preparation objectives and desired cocktail configurations.

Further as shown in the figures 4 10, both single-agent baselines and multi-agent frameworks demonstrated significant challenges in reaching the goal state.

The baseline models showed particularly poor performance in reaching the goal state. None of the single-agent models, including advanced ones like GPT-4o, Claude 3 Opus, and Claude 3.5 Sonnet, successfully achieved the goal state while maintaining legal moves throughout their execution plans except O1-preview. It was able to reach the goal state with valid moves in 10 percent of the cases.

The multi-agent frameworks (Debate, Hierarchical, and Sequential) demonstrated improvements over single-agent approaches, but still struggled significantly: For Debate Framework, Claude 3.5 achieved 50 percent success rate in reaching the goal state and GPT-4o around 30 percent. Further in case of Sequential Framework, Claude 3.5 achieved 30 percent success rate in reaching the goal state and GPT-4o around 20 percent. And

in case of Hierarchical Framework, Claude 3.5 achieved 10 percent success rate in reaching the goal state and GPT-4o around 20 percent.

As shown in figures 3 12 In case of constraint violation, all the models except O1-preview violates constraints in 100 percent of the cases in single-agent framework setting. However O1-preview is relatively better violating constraints in 60 percent of the cases.

For multi-agent framework, there is a huge improvement where with Claude 3.5 constraints are violated only 50 percent of the cases when used with debate framework.

Optimality also follows a similar trend like constraint violation as shown in figures 5 14

The consistently low success rates across both single-agent and multi-agent approaches highlight the particular challenges of the Barman environment. The primary difficulties stem from: Complex state transitions requiring precise sequencing of actions, Need to maintain valid state configurations throughout the execution plan.

5.4 Implications

The evaluation results of our study have significant implications for both the current state and future directions of AI planning systems.

5.4.1 Model Performance Implications

The clear performance hierarchy among models reveals important insights about architectural design and capability scaling. O1-Preview’s superior performance, particularly its 10% success rate in complex environments like Barman while maintaining legal moves, suggests that recent architectural improvements are yielding tangible benefits in complex planning scenarios. The strong showing of Claude 3.5 Sonnet in constraint adherence and goal state interpretation indicates that certain architectural choices may be particularly well-suited for maintaining operational boundaries while pursuing objectives.

The performance gap between Gemini-1.0 Pro and its successor, Gemini-1.5 Flash, provides valuable insights into the impact of iterative model improvements. While the newer version shows marked progress, its continued performance gap behind top models suggests that architectural innovations, rather than just scale, may be crucial for advancing planning capabilities.

5.4.2 Framework Design Implications

Debate Framework Effectiveness

The outstanding performance of the Debate framework carries significant implications for multi-agent system design. The dramatic improvement in GPT-4o’s success rate to 60% while maintaining Claude 3.5 Sonnet’s 95% performance suggests that structured critical evaluation can effectively enhance model reliability. The reduction in constraint violations and achievement of high optimal planning rates (80-95%) indicates that adversarial checking mechanisms may be crucial for robust planning systems.

Sequential Framework Insights

The Sequential framework’s ability to achieve 100% optimality with GPT-4o in feasible cases reveals the potential of iterative refinement approaches. However, its limitations in improving constraint adherence suggest that sequential processing alone may not be sufficient for maintaining operational boundaries. Research Directions The demonstrated success of multi-agent approaches, particularly the Debate framework, suggests promising directions for future research. The performance variations across different game environments indicate the need for domain-adaptive planning strategies. The challenges faced by the Hierarchical framework, especially its 10% success rate with Claude 3.5 in complex environments, highlight the critical need for improved subtask coordination mechanisms.

These findings collectively suggest that future research should focus on developing more sophisticated multi-agent architectures while addressing the specific challenges of subtask coordination and domain adaptation. The success of structured critical evaluation in the Debate framework provides a valuable template for future system designs.

6 Future Work

In our work, we have focused our study on analyzing the performance of several latest single LLMs i.e. GPT-4, GPT-4o, o1-preview, o1-mini, Gemini Pro, Gemini flash, GPT-4, Claude 3.5 Sonnet, Sonar Large (LLama 3.1 70B), Claude 3 Opus etc. vs multi-agent frameworks of them on the prompts we have selected in the dataset. Based on observations, we have identified several promising directions for future work to build upon our current findings. These areas of focus will help us further explore and enhance the planning capabilities of



Figure 1: Plots representing start state interpretation for three games - Single Agent



Figure 2: Plots representing goal state interpretation for three games - Single Agent

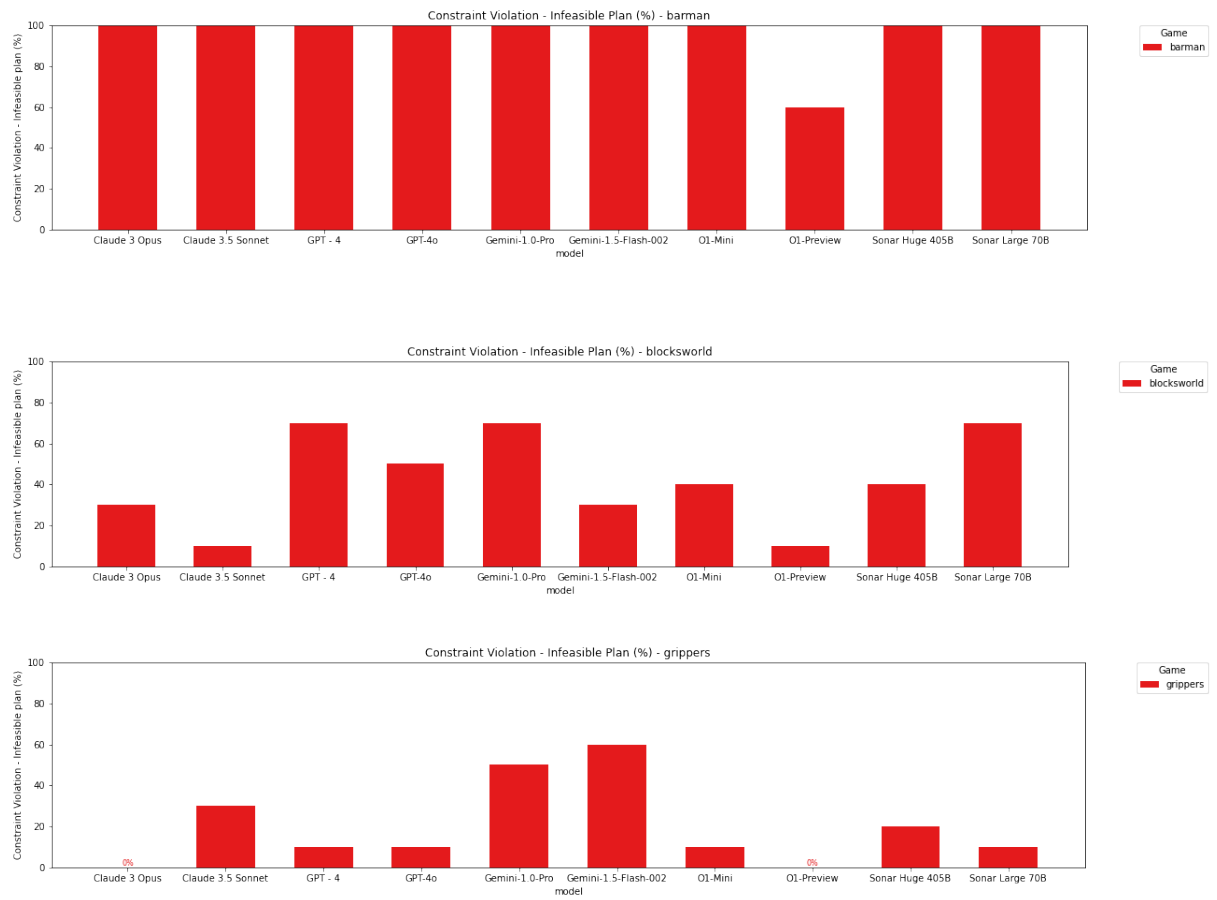


Figure 3: Plots representing constraint violation percentages for three games - Single Agent



Figure 4: Plots representing feasible plan (reached goal state) percentages for three games - Single Agent



Figure 5: Plots representing optimal Solution among feasible plans for three games - Single Agent

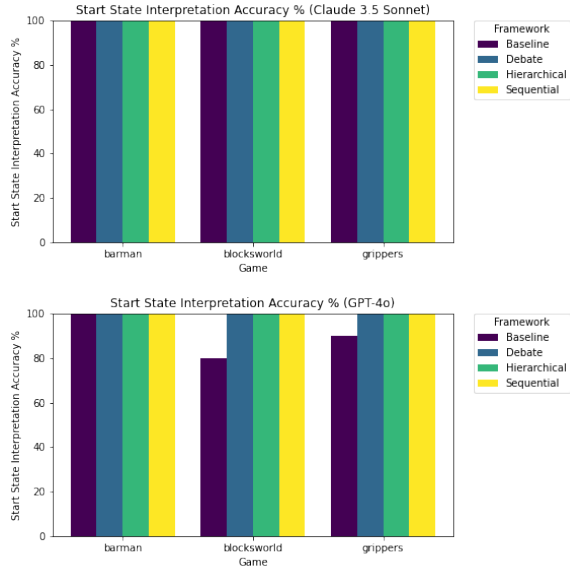


Figure 6: Start State interpretation accuracy of Claude 3.5 and GPT-4o across frameworks, 3 games

	Baseline	Debate	Hierarchical	Sequential
barman	100.0	100.0	100.0	100.0
blocksworld	100.0	100.0	100.0	100.0
grippers	100.0	100.0	100.0	100.0

	Baseline	Debate	Hierarchical	Sequential
barman	100.0	100.0	100.0	100.0
blocksworld	80.0	100.0	100.0	100.0
grippers	90.0	100.0	100.0	100.0

Figure 7: Table of start state interpretation accuracy for three games - Agent frameworks

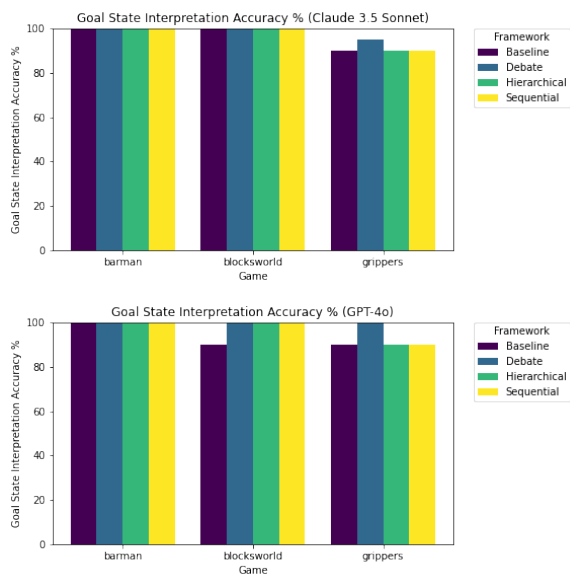


Figure 8: Goal State interpretation accuracy of Claude 3.5 and GPT-4o across frameworks, 3 games

	Baseline	Debate	Hierarchical	Sequential
barman	100.0	100.0	100.0	100.0
blocksworld	100.0	100.0	100.0	100.0
grippers	90.0	95.0	90.0	90.0

	Baseline	Debate	Hierarchical	Sequential
barman	100.0	100.0	100.0	100.0
blocksworld	90.0	100.0	100.0	100.0
grippers	90.0	100.0	90.0	90.0

Figure 9: Table of goal state interpretation accuracy for three games - Agent frameworks

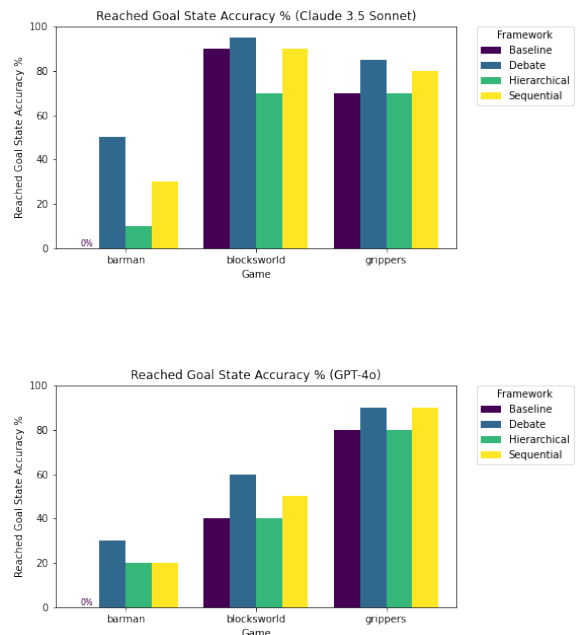


Figure 10: Reached Goal State Success % of Claude 3.5 and GPT-4o across frameworks, 3 games

	Baseline	Debate	Hierarchical	Sequential
barman	0.0	50.0	10.0	30.0
blocksworld	90.0	95.0	70.0	90.0
grippers	70.0	85.0	70.0	80.0

	Baseline	Debate	Hierarchical	Sequential
barman	0.0	30.0	20.0	20.0
blocksworld	40.0	60.0	40.0	50.0
grippers	80.0	90.0	80.0	90.0

Figure 11: Table of reaching goal state success of Claude 3.5 and GPT-4o % for 3 games across frameworks

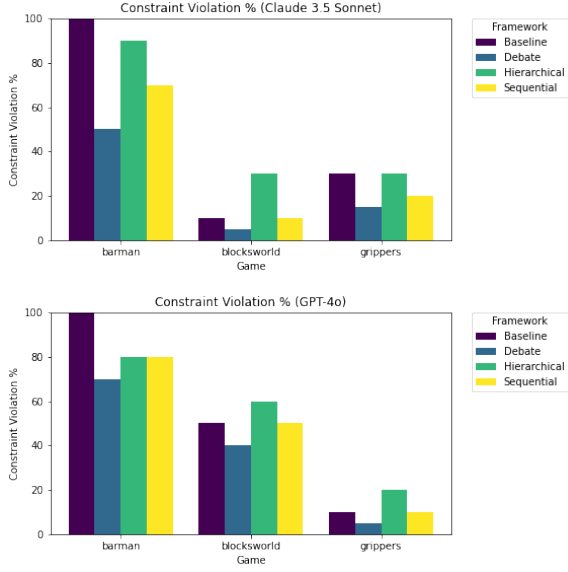


Figure 12: Constraint Violation Rate % of Claude 3.5 and GPT-4o across frameworks, 3 games

	Baseline	Debate	Hierarchical	Sequential
barman	100.0	50.0	90.0	70.0
blocksworld	10.0	5.0	30.0	10.0
grippers	30.0	15.0	30.0	20.0

	Baseline	Debate	Hierarchical	Sequential
barman	100.0	70.0	80.0	80.0
blocksworld	50.0	40.0	60.0	50.0
grippers	10.0	5.0	20.0	10.0

Figure 13: Table of constraint violation rate of Claude 3.5 and GPT-4o % for 3 games across frameworks

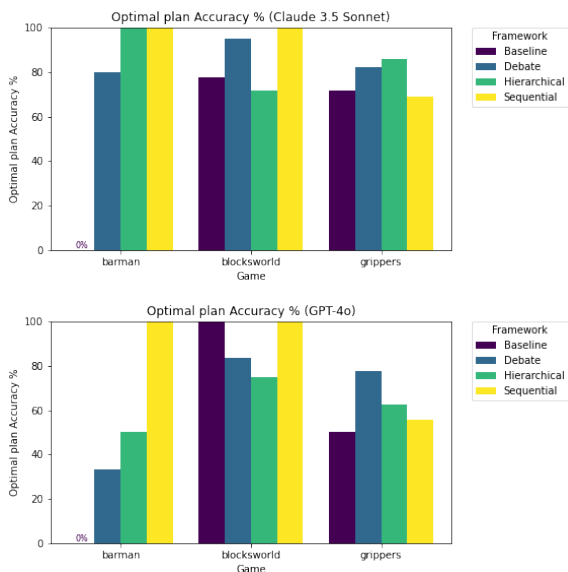


Figure 14: Plots representing optimal solution percentages among feasible plans for three games - Multi Agent

	Baseline	Debate	Hierarchical	Sequential
barman	0.0	80.0	100.0	100.0
blocksworld	77.78	94.74	71.43	100.0
grippers	71.43	82.35	85.71	68.75

	Baseline	Debate	Hierarchical	Sequential
barman	0.0	33.33	50.0	100.0
blocksworld	100.0	83.33	75.0	100.0
grippers	50.0	77.78	62.5	55.56

Figure 15: Table of optimal solutions percentage among feasible plans of Claude 3.5 and GPT-4o % for 3 games across frameworks

LLMs in structured environments and better evaluation of these tasks. Some of the future directions are:

6.1 Single LLM performance improvement

We have used the prompts from the dataset we have mentioned without any modification. However, we observed that few of the prompts lacked specific definitions like optimality can be minimum number of total steps, overall execution time which can be reduced with parallel execution with many steps. So it is worthy to explore understand the human baseline against these prompts, iteratively improve the prompts for better evaluation.

- **Prompt Tuning:** Exploring several ways of defining the game in different templates and restructuring the prompt based on the family of the model instead of single prompt for all the models. It has been observed that prompts designed in fashion similar to training format have provided better model performance. There is a possibility to develop a model for converting the prompt for different family of models and which can also be used to improve the prompt for a specific model for better plan.
- **Reflection Mechanisms:** Implementing a self-reflection stage where the model evaluates its own plan before finalizing it. This could involve asking the model to identify potential flaws or inefficiencies in its initial plan and make improvements as a reflection on its own thinking.
- **Constraint Verification Prompting:** In our research work, ablation studies has shown that models perform better when explicitly prompted to consider constraints. So developing a series of targeted prompts that require

the model to actively verify its adherence to game rules and environmental constraints at each step of the planning process. This approach aims to reduce constraint violations and improve overall plan validity.

6.2 Automatic planning evaluator

We had to spend significant amount of time in evaluating the output for the metrics as there is a possibility to have multiple optimal plans to complete the task and sometimes the model combines multiple actions as a single action. So, it will be extremely beneficial to develop an automated evaluator which can verify the plan for the metrics instead of manual evaluation which can speed up the evaluation of the proposed plan, thus the research in this space.

6.3 Other Multi-agent Frameworks

In our study, we had explored three specific multi-agent frameworks: multi-agent debate, iterative multi-agent framework, and hierarchical collaborative multi-agent framework. We can also explore additional multi-agent frameworks such as Mixture Of Agents, Persona-based multi-agent framework, Hierarchical debate multi-agent framework, Multi-Agent Reinforcement Learning (MARL), etc. These frameworks can improve the planning of the LLMs.

6.4 Generalizability to Other Tasks

To assess the broader applicability of our findings, we plan to extend our research beyond the current game environments to real-world planning scenarios. This expansion will help evaluate the generalizability of our multi-agent planning frameworks and provide insights into their potential for solving complex problems in practical domains.

Contributions

Our team of three collaborated closely to establish baselines for single-agent models across three game environments and the three multi-agent frameworks. The work was distributed as follows:

- Sri Harsha Gollamudi: Blocksworld and Debate multi-agent framework
- Samavedam Manikhanta Praphul: Grippers and Hierarchical multi-agent framework
- Poorna Chandra Vemula: Barman and Iterative multi-agent framework

Each team member was responsible for evaluating the performance of all selected single-agent models (GPT-4 Omni, Sonar Large, Sonar Huge, Claude 3 Opus, and Claude 3.5 Sonnet) on their assigned games.

We equally contributed to the literature review and methodology sections of this paper, ensuring a comprehensive understanding of the current state of LLM planning capabilities and the potential of multi-agent frameworks. This collaborative approach allowed us to develop a robust set of evaluation metrics and experimental design.

The team worked together on the final report, combining our individual findings and insights to present a cohesive analysis of LLM planning performance across different game environments and model architectures.

References

- AI Anthropic. 2024a. Claude 3.5 sonnet model card addendum. *Claude-3.5 Model Card*.
- AI Anthropic. 2024b. Claude 3.5 sonnet model introduction news. <https://www.anthropic.com/news/claude-3-5-sonnet>.
- Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology*, 15(3):1–45.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. 2023. Improving factuality and reasoning in language models through multi-agent debate. *arXiv preprint arXiv:2305.14325*.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. 2022. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*, pages 9118–9147. PMLR.
- Kevin Lin, Christopher Agia, Toki Migimatsu, Marco Pavone, and Jeannette Bohg. 2023. Text2motion: From natural language instructions to feasible plans. *Autonomous Robots*, 47(8):1345–1365.
- Tongxuan Liu, Xingyu Wang, Weizhe Huang, Wenjiang Xu, Yuting Zeng, Lei Jiang, Hailong Yang, and Jing Li. 2024. Groupdebate: Enhancing the efficiency of multi-agent debate using group discussion. *arXiv preprint arXiv:2409.14051*.

Zhao Mandi, Shreeya Jain, and Shuran Song. 2024. Roco: Dialectic multi-robot collaboration with large language models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 286–299. IEEE.

OpenAI. 2024a. Hello gpt-4o. <https://openai.com/index/hello-gpt-4o/>. Released: 2024-05-13.

OpenAI. 2024b. Learning to reason with llms. <https://openai.com/index/learning-to-reason-with-llms/>. Accessed: 2024-10-27.

Perplexity AI. 2024a. Sonar large (405b) model. <https://docs.perplexity.ai/guides/model-cards>.

Perplexity AI. 2024b. Sonar large (70b) model. <https://docs.perplexity.ai/changelog/changelog#introducing-new-and-improved-sonar-models>. Released: 2024-07.

Chan Hee Song, Jiaman Wu, Clayton Washington, Brian M Sadler, Wei-Lun Chao, and Yu Su. 2023. Llm-planner: Few-shot grounded planning for embodied agents with large language models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 2998–3009.

Kevin Wang, Junbo Li, Neel P Bhatt, Yihan Xi, Qiang Liu, Ufuk Topcu, and Zhangyang Wang. 2024. On the planning abilities of openai’s o1 models: Feasibility, optimality, and generalizability. *arXiv preprint arXiv:2409.19924*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2024. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36.

A Appendix: Prompts used for game briefing

This appendix section contains the prompt used at the beginning for LLM to provide the rules of the game clearly covering aspects of the game.

A.1 BlocksWorld

The robot has four actions: pickup, put-down, stack, and unstack. The domain assumes a world where there are a set of blocks that can be stacked on top of each other, an arm that can hold one block at a time, and a table where blocks can be placed. The actions defined in this domain include:

pickup: allows the arm to pick up a block from the table if it is clear and the arm is empty. After the pickup action, the arm will be holding the block, and the block will no longer be on the table or clear.

putdown: allows the arm to put down a block on the table if it is holding a block. After the putdown action, the arm will be empty, and the block will be on the table and clear.

stack: allows the arm to stack a block on top of another block if the arm is holding the top block and the bottom block is clear. After the stack action, the arm will be empty, the top block will be on top of the bottom block, and the bottom block will no longer be clear.

unstack: allows the arm to unstack a block from on top of another block if the arm is empty and the top block is clear. After the unstack action, the arm will be holding the top block, the top block will no longer be on top of the bottom block, and the bottom block will be clear.

A.2 Barman

You are a robot barman that manipulates drink dispensers, shot glasses and a shaker. You have two hands. The goal is to find a plan that serves a desired set of drinks. Here are the actions you can do

Grasp a container

Leave a container on the table

Fill a shot glass with an ingredient

Refill a shot glass with an ingredient

Empty a shot glass

Clean a shot glass

Pour an ingredient from a shot glass to a clean shaker

Pour an ingredient from a shot glass to a used shaker

Empty a shaker

Clean a shaker

Shake a cocktail in a shaker

Pour from a shaker to a shot glass

You have the following restrictions on your actions:

You can only grasp a container if your hand is empty and it is on the table.

You can only leave a container if you are holding it.

You can only fill a shot glass if you are holding the shot glass, your other hand is empty, the shot glass is empty and clean.

You can only refill a shot glass if you are holding the shot glass, your other hand is empty, the shot glass is empty and has contained the same ingredient before.

You can only empty a shot glass if you are holding the shot glass and it contains a beverage.

You can only pour from a shot glass to a clean shaker if you are holding the shot glass, the shot glass contains an ingredient, and the shaker is empty and clean.

You can only pour from a shot glass to a used shaker if you are holding the shot glass, the shot glass contains an ingredient, the shaker is unshaken and at a level not full.

You can only empty a shaker if you are holding the shaker and the shaker contains a shaken beverage.

You can only clean a shaker if you are holding the shaker, your other hand is empty, and the shaker is empty.

You can only shake a cocktail if you are holding the shaker, your other hand is empty, the shaker is unshaken, and the shaker contains two ingredients, and both ingredients are parts of a cocktail.

You can only pour from a shaker to a shot glass if you are holding the shaker, the shaker contains the cocktail, the shaker is shaken, and the shot glass is empty and clean.

Once you grasp a container, you are holding the container and the container is not on the table.

Once you leave a container on the table, your hand becomes empty.

Once you pour an ingredient from a shot glass to a shaker, the shaker contains the ingredient and is at one level above

the previous level, and the shot glass becomes empty.

Once you empty a shaker, the shaker is at the empty level.

Once you shake, the two ingredients in the shaker become a cocktail.

Once you pour from a shaker to a shot glass, the shot glass contains the beverage in the shaker, the shot glass is no longer clean and empty, and the shaker is at one level below the previous level.

A.3 Grippers

You are a robot with a gripper that can move objects between different rooms. There are three actions defined in this domain:

The move action: This action allows the robot to move from one room to another. The action has a single precondition, which is that the robot is currently in a room. The effect of this action is to move the robot to another room and to remove the fact that it is in the original room.

The pick action: This action allows the robot to pick up an object using the gripper. The action has three preconditions: (1) the object is located in a room (2) the robot is currently in the same room and (3) the gripper is free (i.e., not holding any object). The effect of this action is to update the state of the world to show that the robot is carrying the object using the gripper, the object is no longer in the room, and the gripper is no longer free.

The drop action: This action allows the robot to drop an object that it is carrying. The action has two preconditions: (1) the robot is currently carrying the object using the gripper, and (2) the robot is currently in a room. The effect of this action is to update the state of the world to show that the robot is no longer carrying the object using the gripper, the object is now located in the room, and the gripper is now free.

A.4 Multi-Agent Debate Framework Example

Multi-Agent Debate Framework

Agent 1 (Round 1 Plan):

- Unstack b6 from b3
- Unstack b4 from b7
- Unstack b2 from b5
- Stack b5 on b6
- Stack b2 on b5
- Stack b3 on b2
- Stack b7 on b3
- Stack b1 on b7
- Stack b4 on b1

Agent 2 (Round 2 Critique of

Agent 1):

Agent 1's plan is close, but it's not optimal and has a few issues:

- You don't need to unstack b2 from b5 as it's already in the correct position.
- You're moving b1 unnecessarily. It should stay on the table.
- The order of stacking is not optimal, leading to more moves than necessary.

Agent 2 (Round 2 Plan):

Here's a more efficient plan:

- Unstack b6 from b3
- Unstack b4 from b7
- Stack b6 on the table
- Stack b5 on b6
- Stack b3 on b2 (b2 is already on b5)
- Stack b7 on b3
- Stack b4 on b1 (b1 is already on the table)

Figure 16: Example output of Multi-Agent Debate Framework showcasing how the plan is improved